



REKURSIF

Lutfi Hakim, S.Pd., M.T.



D4 - TEKNOLOGI REKAYASA PERANGKAT LUNAK
POLITEKNIK NEGERI BANYUWANGI

Overview

- Definisi Rekursif
- Kelebihan dan Kekurangan
- Bentuk Umum
- Implementensi studi kasus dan pada Dart
- Tail & Non-Tail Recursion

Definisi Rekursif

- Merupakan suatu fungsi atau prosedur
- Fungsi yang berisi definisi dirinya sendiri
- Fungsi yang memanggil dirinya sendiri
- Proses terjadinya berulang-ulang
- Perlu memperhatikan “kondisi berhenti”

Kelebihan & Kekurangan

Kelebihan

- Program lebih singkat
- Beberapa kasus lebih mudah dan efektif diselesaikan secara rekursif

Catatan: Jika sebuah masalah bisa diselesaikan secara iteratif, maka gunakan iteratif

Kekurangan

- Membutuhkan memori lebih besar, setiap kali bagian dirinya dipanggil, ada memori yang dibutuhkan
- Rekursif seringkali tidak bisa berhenti sehingga memori habis terpakai dan program hang
- Program menjadi sulit dibaca

Bentuk Umum Fungsi Rekursif

```
TipeDataKembalian nama_fungsi(daftar parameter) {  
    ...  
    nama_fungsi(daftar_parameter)  
}
```

Faktorial: Rekursif

- Konsep Faktorial
 $n! = n(n-1)(n-2) \dots 1$
- Dapat diselesaikan dengan
 - Cara Biasa
 - Rekursif

Iterative factorial:

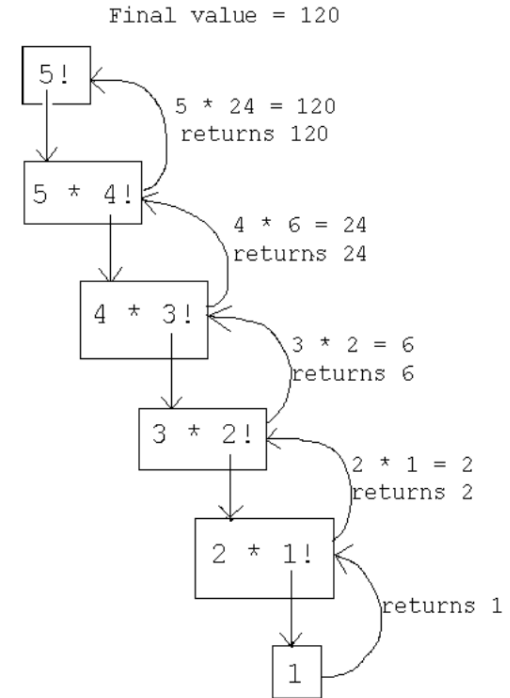
$$n! = \prod_{k=1}^n k$$

Recursive factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0, \\ (n-1)! \times n & \text{if } n > 0. \end{cases}$$

Faktorial: Rekursif

- $n!$ adalah hasil kali n dengan $(n-1)!$
- Untuk menyelesaikan $(n-1)!$ Sama dengan $n!$, yaitu $n-1$ dikalikan dengan $(n-2)!$, dan $(n-2)!$ Adalah $n-2$ dikalikan dengan $(n-3)!$, dan seterusnya sampai dengan $n=1$
- Saat $n=1$, kita menghentikan perulangan (kondisi berhenti / stopping role)



Contoh 1: Menghitung Faktorial

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$4! = 4 \times 3 \times 2 \times 1$$

Dengan kata lain, $5! = 5 \times 4!$

Metode Iteratif:

Menggunakan loop yang mengalikan masing-masing bilangan dengan hasil sebelumnya. Dapat didefinisikan dengan persamaan berikut:

$$n! = (n) (n-1) (n-2) \dots (1)$$

Perhitungan

$$n! = 1$$

$$n! = n * (n-1)!$$

$$0! = 1$$

$$1! = 1 * (1-1)!$$

$$= 1 * 0!$$

$$= 1 * 1 = 1$$

$$2! = 2 * (2-1)!$$

$$= 2 * 1!$$

$$= 2 * 1 = 2$$

Jika $n = 0$ stopping role, anchor

Jika $n > 0$ Langkah induktif(memanggil dirinya sendiri dengan nilai parameter yang berbeda)

Faktorial: Cara Biasa

```
int faktorialIteratif(int n) {  
    if (n < 0) return -1;  
    else if (n > 1) {  
        int faktorial = 1;  
        for (int i = n; i > 0; i--) {  
            faktorial *= i;  
        }  
        return faktorial;  
    } else {  
        return 1;  
    }  
}
```

```
void main() {  
    print(faktorialIteratif(3));  
}
```

Faktorial: Implementasi Rekursif

```
int factorialRekursif(int n) {  
    if (n == 0) return 1;  
    else return n * factorialRekursif(n - 1);  
}  
  
void main() {  
    print(factorialRekursif(3));  
}
```

Contoh 2: Deret Fibonacci

- Deret Fibonacci adalah suatu deret matematika, yang berasal dari jumlah dua bilangan sebelumnya
- Leonardo Fibonacci berasal dari Italia 1170-1250
- Deret Fibonacci $f_1, f_2, \dots$, didefinisikan secara rekursif sebagai berikut:

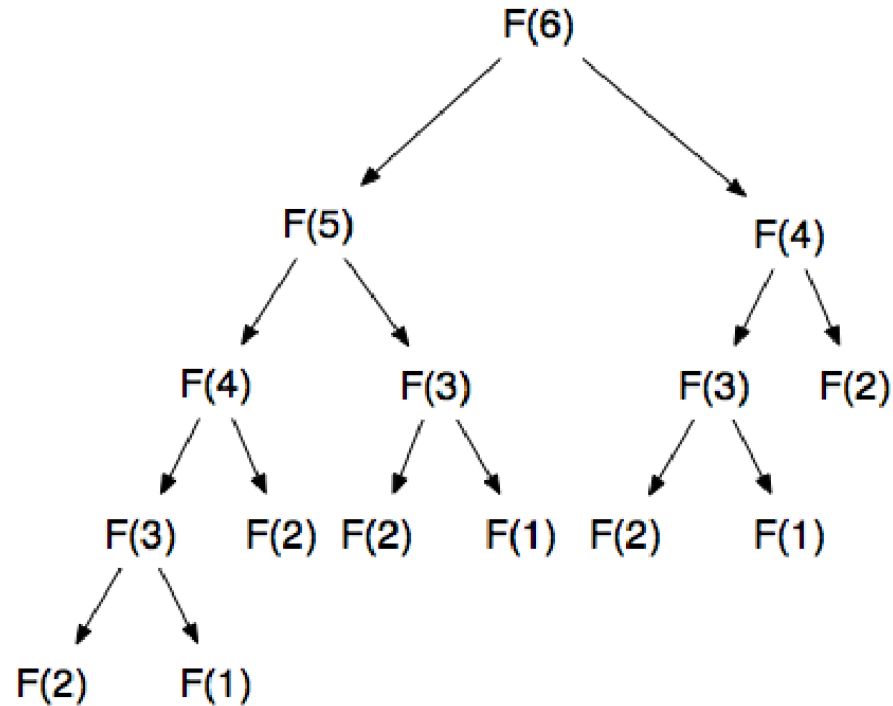
$$f_1 = 1$$

$$f_2 = 1$$

$$f_n = f_{n-1} + f_{n-2} \text{ for } n \geq 3$$

- Deret: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, ...

Deret Fibonacci



Deret Fibonacci

- Metode Iteratif:
Mulai dari suku ke-3 menggunakan perulangan, dimana bilangan Fibonacci adalah jumlah bilangan pertama dengan bilangan kedua, yang setelah dijumlahkan. Selanjutnya bilangan kedua akan disimpan sebagai bilangan kesatu, dan hasil Fibonacci akan disimpan sebagai bilangan kedua. Untuk suku ke-1 dan 2, hanya akan menghasilkan angka 1

Implementasi pada Dart

```
int fibonacciIteratif(int n) {  
    int fibo = 0, f1 = 1, f2 = 1;  
    if (n == 1 || n == 2) {  
        return 1;  
    } else {  
        for (int i = 3; i <= n; i++) {  
            fibo = f1 + f2;  
            f1 = f2;  
            f2 = fibo;  
        }  
    }  
    return fibo;  
}  
  
void main() {  
    print(fibonacciIteratif(3));  
}
```

Fibonacci: Rekursif

- Fibonacci suku ke- n adalah hasil penjumlahan Fibonacci ($n-1$) dan Fibonacci ($n-2$)
- Untuk menyelesaikan Fibonacci ($n-1$) sama dengan penjumlahan dari Fibonacci ($n-2$) dan Fibonacci ($n-3$), dan seterusnya
- Kondisi berhenti, saat nilai $n = 1$ atau $n = 2$, nilai dari Fibonacci adalah 1

Implementasi pada Dart

```
int fibonacciRekursif(int n) {  
    if (n == 1 || n == 2) {  
        return 1;  
    } else {  
        return fibonacciRekursif(n - 1) + fibonacciRekursif(n - 2);  
    }  
}  
  
void main() {  
    print(fibonacciRekursif(5));  
}
```

Evaluasi Fibonacci Rekursif

- Untuk $N = 40$, fungsi melakukan >300juta pemanggilan rekursif. $F_{40} = 102.334.155$
- Berat!
- Himbauan: Jangan membiarkan ada duplikasi proses yang mengerjakan input sama pada pemanggilan rekursif yang berbeda
Misal: $f(3)$ dipanggil saat menyelesaikan $f(4)$ dan $f(5)$
- Ide: Simpan nilai Fibonacci yang sudah dihitung dalam sebuah list -> Dynamic Programming
- Dynamic Programming : Menyelesaikan sub permasalahan dengan menyimpan hasil sebelumnya

Fibonacci: Dynamic Programming

```
List<int> result = [];  
  
int fibonacciDP(int n) {  
    result = List<int>.filled(n + 1, 0);  
    if (n <= 1)  
        return 1;  
    else {  
        result[0] = 1;  
        result[1] = 1;  
        for (int i = 2; i <= n; i++) {  
            result[i] = result[i - 1] + result[i - 2];  
        }  
        return result[n];  
    }  
}  
  
void main() {  
    int nilai = 40;  
    print(fibonacciDP(nilai - 1));  
}
```

Tower Hanoi

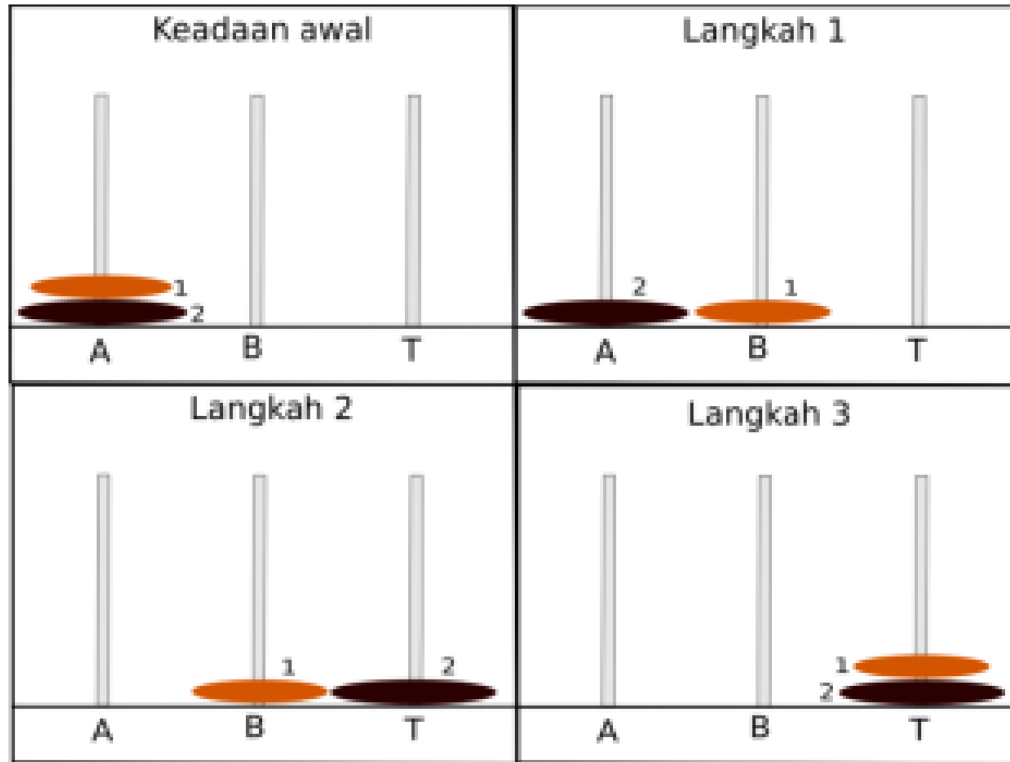
- Tower Hanoi adalah permainan puzzle dengan tiga tiang dan sejumlah disk/cakram yang tersusun pada tiang
- Disk memiliki ukuran yang bervariasi. Disk-disk tertumpuk rapi berurutan berdasarkan ukurannya dalam salah satu tiang, disk terkecil diletakkan teratas, sehingga membentuk kerucut
- Tujuan adalah memasukkan semua disk dari tiang awal ke tiang tujuan dengan menggunakan tiang bantuan dengan aturan:
 - Hanya satu disk dapat dipindahkan pada suatu waktu
 - Sebuah disk tidak dapat ditempatkan di atas disk yang lebih kecil

Tower Hanoi

Oleh Edouard Lucas abad 19

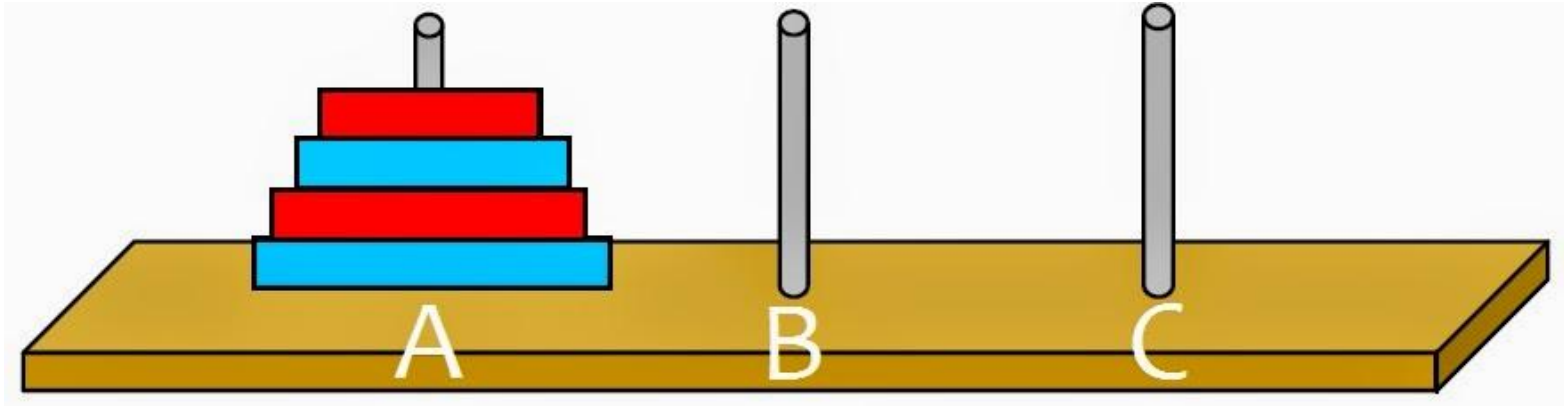
- Seorang biarawan memiliki 3 Menara
- Diharuskan memindahkan 64 piringan emas
- Diameter piringan tersebut tersusun dari ukuran kecil ke besar
- Biarawan berusaha memindahkan semua piringan dari Menara pertama ke Menara ketiga tetapi harus melalui Menara kedua sebagai Menara tampungan
- Kondisi:
 - Piringan tersebut hanya bisa dipindahkan satu persatu
 - Piringan terbesar tidak bisa diletakkan di piringan yang lebih kecil
- Secara teori, diperlukan $2^{64}-1$ perpindahan. Jika kita salah memindahkan, maka jumlah perpindahan akan lebih banyak lagi.

Tower Hanoi (2)



- Jika terdapat 3 piringan, ada berapa kali jumlah perpindahan?

Tower Hanoi (3)



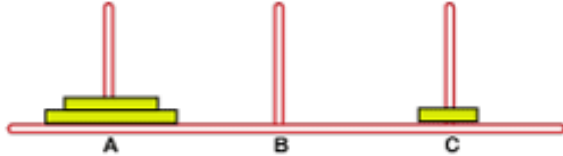
Jika terdapat 3 piringan, ada berapa kali jumlah perpindahan?

Tower Hanoi (4)

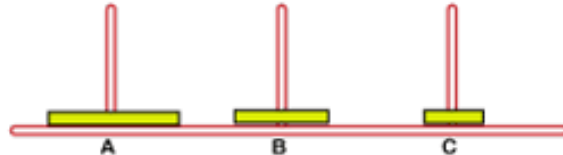
Move - 0



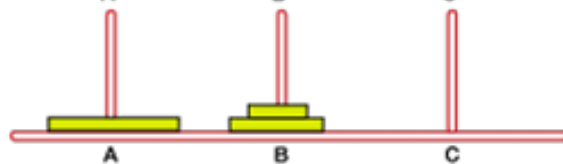
Move - 1



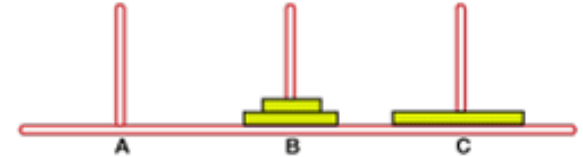
Move - 2



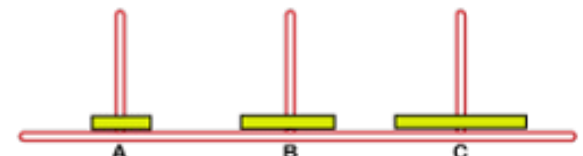
Move - 3



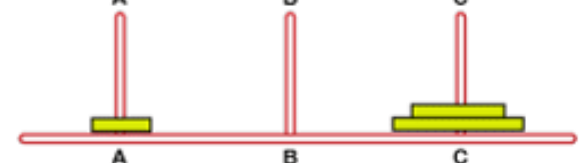
Move - 4



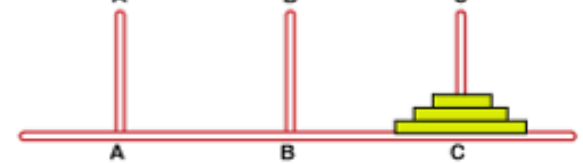
Move - 5



Move - 6

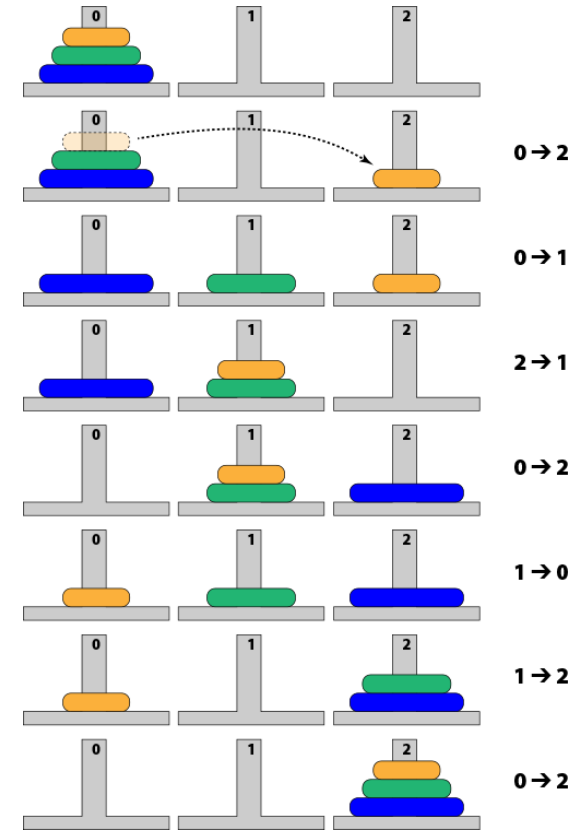


Move - 7



Algoritma Tower Hanoi

- Jika $n == 1$, pindahkan piringan dari A ke C
- Jika tidak:
 - Pindahkan $n-1$ piringan dari A ke B menggunakan C sebagai tampungan
 - Pindahkan $n-1$ piringan dari B ke C menggunakan A sebagai tampungan



Towers of Hanoi

- Jumlah perpindahan disk bertambah secara exponential dengan bertambahnya jumlah disk
- Solusi secara rekursif lebih sederhana dibandingkan dengan solusi iterative
- Simulasi Tower Hanoi:
<https://www.mathsisfun.com/games/towerofhanoi.html>

Implementasi pada Dart

```
import 'dart:io';

void towerOfHanoi(int n, String sumber, String tujuan, String bantuan) {
  if (n == 1) {
    print('Pindahkan cakram 1 dari $sumber ke $tujuan');
    return;
  } else {
    towerOfHanoi(n - 1, sumber, bantuan, tujuan);
    print('Pindahkan cakram $n dari $sumber ke $tujuan');
    towerOfHanoi(n - 1, bantuan, tujuan, sumber);
  }
}

void main() {
  var input = stdin;
  stdout.write("Masukkan Jumlah Cakram/Disc: ");
  int cakram = int.parse(input.readLineSync());
  towerOfHanoi(cakram, 'A', 'B', 'C');
  print("Selamat, cakram selesai terpindahkan ke tiang tujuan!");
}
```

Tail Recursive Function

- Jika hasil akhir yang akan dieksekusi berada dalam tubuh fungsi
- Tidak memiliki aktivitas selama fase balik
- Metode rekursif:
 - Tail Recursive
 - Nontail Recursive
- Method tail recursive memiliki pemanggilan recursive di akhir method
- Recursive methods yang bukan tail recursive disebut non-tail recursive

Apa itu Factorial Tail Recursive?

- Apakah method factorial adalah tail recursive?

```
void tail(int i) {  
    if (i > 0) {  
        print('$i ');  
        tail(i - 1);  
    }  
}  
  
void main() {  
    tail(5); // Ganti dengan nilai yang diinginkan  
}
```

- Bukan tail recursive, karena pada saat kembali dari recursive call $x * \text{fact}(x-1)$, masih terdapat operasi perkalian

Factorial Tail Recursive?

- Apakah method `prog()` adalah tail recursive?

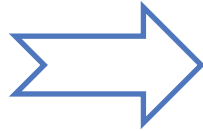
```
void nonProg(int i) {  
    if (i > 0) {  
        nonProg(i - 1);  
        print('$i ');  
        nonProg(i - 1);  
    }  
}  
  
void main() {  
    nonProg(5); // Ganti dengan nilai yang diinginkan  
}
```

- Tidak, karena dibaris awal terdapat recursive call
- Pada tail recursive, recursive call menjadi statemen akhir, dan tidak ada recursive call di atasnya

Advantage of Tail Recursive Method

- Method dengan Tail Recursive, mudah dirubah menjadi iterative

```
void tail(int i) {  
    if (i > 0) {  
        print('$i ');  
        tail(i - 1);  
    }  
}  
  
void main() {  
    tail(5);  
}
```



```
void iterative(int i) {  
    for (; i > 0; i--) {  
        print('$i ');  
    }  
}  
  
void main() {  
    iterative(5);  
}
```

- Smart compilers can detect tail recursion and convert it to iterative to optimize code
- Used to implement loops in languages that do not support loop structures explicitly (e.g. prolog)

Implementasi Deret Fibonacci

```
class Fibonacci {  
    static int fib(int n) {  
        if (n <= 1) return n;  
        else return fib(n - 1) + fib(n - 2);  
    }  
}  
  
void main(List<String> args) {  
    int N = int.parse(args[0]);  
    for (int i = 1; i <= N; i++) {  
        print('$i: ${Fibonacci.fib(i)}');  
    }  
}
```


Converting Non-Tail to Tail Recursive

- Method non-tail recursive dapat diubah menjadi tail-recursive method dengan menambahkan parameter tambahan untuk menampung hasil.

method `fact(int n)` → `fact_aux(int n, int result)`

- Teknik yang digunakan biasanya membuat fungsi tambahan (method `fact(int n)`) yang memanggil method tail recursive

```
int fact_aux(int n, int result) {  
    if (n == 1)  
        return result;  
    return fact_aux(n - 1, n * result)  
}  
int fact(n) {  
    return fact_aux(n, 1);  
}
```

Rekursif Tail: Factorial()

$$F(n, a) = \begin{cases} a & \text{jika } n = 0, n = 1 \\ F(n - 1, na) & \text{jika } n > 1 \end{cases}$$

$F(4,1) = F(3,4)$	Fase awal
$F(3,4) = F(2,12)$	
$F(2,12) = F(1,24)$	
$F(1,24) = 24$	Kondisi Terminal
24	Fase Balik Rekursif Lengkap

Converting Non-Tail to Tail Recursive

- Method tail-recursive Fibonacci diimplementasikan menggunakan dua parameter bantuan untuk menampung hasil

auxiliary parameters!

```
int fib_aux ( int n , int next, int result)
{
    if (n == 0)
        return result;
    return fib_aux(n - 1, next + result, next);
}
```

- Untuk menghitung `fib(n)`, call `fib_aux(n, 1, 0)`

Review Questions

- Apakah Rekursif = Dynamic Programming? Jika sama, apa persamaannya, jika beda, apa perbedaannya?
- Seberapa pentingkah menentukan kondisi berhenti pada sebuah fungsi rekursif?
- Menurut anda, apakah konsep rekursif juga dapat diterapkan pada sebuah prosedur?



Tutorial Video

- Fungsi Rekursif:
<https://www.youtube.com/watch?v=wdrSmK18nj4&list=PLZS-MHyEIRo51w0Hmqi0C8h2KWNzDfo6F&index=45>
- Rekursif Bercabang:
<https://www.youtube.com/watch?v=TsUMDFJEx7I&list=PLZS-MHyEIRo51w0Hmqi0C8h2KWNzDfo6F&index=46>

Referensi

- Hakim, Lutfi; Kristanto, Sepyan Purnama; Yusuf, Dianni. 2026, “Buku Ajar Struktur Data: Konsep dan Implementasi di Bahasa Pemrograman Dart”. Banyuwangi: Poliwangi Press
- Sande, Jonathan; Lau, Kevin; Ngo, Vincent. 2022. “*Data Structures & Algorithms in Dart (First Edition): Implementing Practical Data Structures in Dart*”. Kodeco Inc.
- Goodrich, Michael T.; Ramassia, Roberto; Goldwasser, Michael H. 2014. “Data Structures & Algorithms in Java”. United State of America: Wiley
- <https://www.geeksforgeeks.org/data-structures/>
- <https://www.kodeco.com/books/data-structures-algorithms-in-dart/v1.0/chapters/3-basic-data-structures-in-dart>
- https://www.tutorialspoint.com/dart_programming/dart_programming_lists.htm
- <https://www.darttutorial.org/dart-tutorial/dart-list/>

A large, stylized map of Indonesia is formed by a collection of blue dots of varying sizes, scattered across the center of the slide. The dots are more densely packed in some areas, creating a pixelated or mosaic effect that outlines the archipelago.

TERIMA KASIH